

DHBW Stuttgart
Campus Horb

Studienarbeit

Titel der Studienarbeit

Untertitel (optional)

Studiengang:	Informatik
Kurs:	TINF23A
Autor:	Max Mustermann
Matrikelnummer:	1234567
Unternehmen:	Musterfirma GmbH
Erstpruefer:	Prof. Dr. Vorname Nachname
Zweitpruefer:	Vorname Nachname
Abgabedatum:	16.03.2026

Inhaltsverzeichnis

1 Einleitung

1.1 Motivation

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

1.2 Zielsetzung

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

1.3 Aufbau der Arbeit

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel

magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

- Was ist Gravitax und wie funktioniert es? - Kommunikation, Protokoll, Libraries etc. - Analyse der Library mit Verbesserungsvorschlägen - Umsetzung von 2/3 Murmelbahnen mit samt Code - Analyse ob nutzbar um damit wissen zu vermitteln bzw. als Lernplattform

2 Grundlagen

2.1 Technische Grundlagen

2.1.1 GraviTrax und GraviTrax POWER

GraviTrax ist ein modulares Marmelbahnsystem der Ravensburger-Verlag GmbH, das aus geometrisch kompatiblen hexagonalen Bausteinen besteht und eine freie Konfiguration von Bahnverläufen erlaubt (siehe Abbildung ??). Die Standardbausteine (z.B. Geraden, Kurven, Weichen und Spiralen) werden durch sogenannte „Action“-Elemente erweitert, welche aktiv in den Bewegungsablauf eingreifen und damit komplexere Streckenlogiken ermöglichen (beispielsweise magnetische Kanonen, Katapulte oder Seilrutschen-Elemente; vgl. Abbildung ?? und Abbildung ??).



Abbildung 2.1: Beispiel eines GraviTrax-Marmelbahnlaufs mit mehreren Bausteinen und Aktorelementen

Mit der Produktlinie „GraviTrax POWER“ wird das System um batteriebetriebene, elektronische Bausteine ergänzt (siehe Abbildung ??). Diese POWER-Elemente können

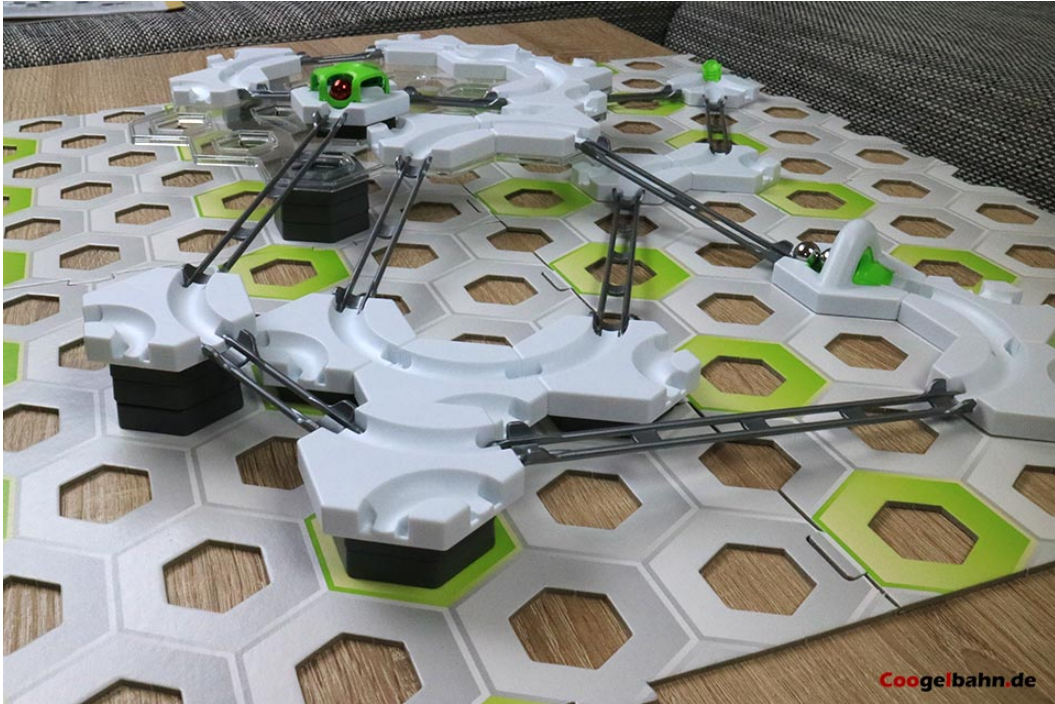


Abbildung 2.2: Aufreihung von Basis GraviTrax-Bausteinen (Geraden, Kurven, Weichen)

Zustände erfassen, Ereignisse auslösen oder auf Signale anderer Steine reagieren. Aus Sicht dieser Arbeit sind insbesondere (i) Trigger-Elemente relevant, die Ereignissignale erzeugen, sowie (ii) Aktor- bzw. Schaltelemente (z. B. „Switches“), die auf diese Signale reagieren und dadurch den Bahnverlauf oder den Systemzustand beeinflussen.

2.1.2 Kommunikationsarchitektur: BLE-Backhaul und 2,4 GHz-Funk

Die POWER-Elemente kommunizieren untereinander über ein proprietäres Funkprotokoll im lizenzfreien 2,4 GHz-ISM-Band. Diese Funkkommunikation ist grundsätzlich unabhängig von einer externen Bridge: Ereignisse werden von einem sendenden Stein über Funk ausgestrahlt und können von allen in Reichweite befindlichen POWER-Steinen empfangen werden.

Der „Power Connect“-Stein nimmt in diesem System eine optionale Gateway-Rolle ein. Er dient primär dazu, eine Verbindung „nach außen“ herzustellen, indem er ein externes Endgerät (z. B. Smartphone oder Computer) über Bluetooth Low Energy (BLE) anbindet und eingehende Steuerinformationen in Funkpakete übersetzt. Praktisch bedeutet dies: Anwendungen wie die GraviTrax-App können über BLE mit dem Power Connect gekoppelt werden, um anschließend Ereignisse in das Funknetz der POWER-Elemente einzuspeisen. Für rein interne Interaktionen zwischen POWER-Steinen (ohne

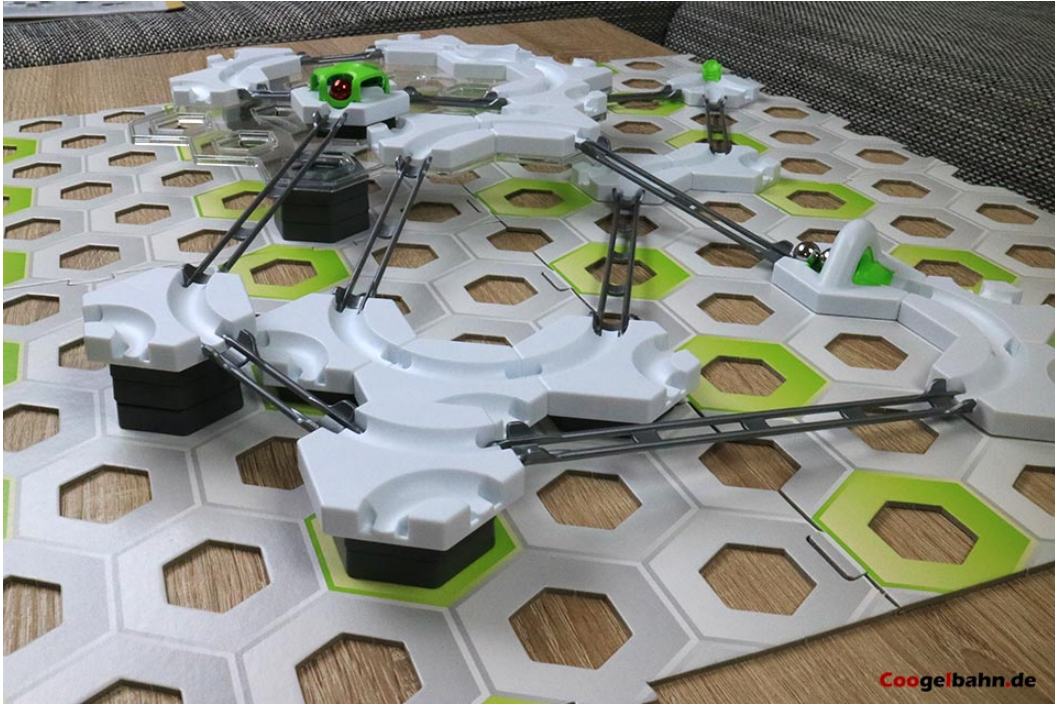


Abbildung 2.3: Aufreihung von Action-Elementen (z. B. magnetische Kanone, Katalpult, Seilrutsche)

externe Steuerung) ist der Power Connect hingegen nicht erforderlich.

Neben diesem offenen Broadcast-Betrieb existiert ein restriktiver Betriebsmodus, der in der offiziellen Programmierschnittstelle als „Bridge Only mode“ bezeichnet wird. In diesem Modus berücksichtigen POWER-Elemente ausschließlich Funkpakete, die als von einer Bridge (Power Connect) stammend gekennzeichnet sind; Signale anderer Steine werden verworfen. Die Aktivierung erfolgt durch das Senden eines LOCK-Statuscodes, die Deaktivierung durch einen UNLOCK-Statuscode (jeweils im entsprechenden Farbkanal). Ob ein Element im Bridge Only mode ist, lässt sich an der Farbkanal-LED erkennen: Im normalen Modus leuchtet die LED in der Farbe des Kanals (Rot, Grün oder Blau), im Bridge Only mode blinkt diese periodisch. Konzeptionell handelt es sich damit um eine Protokollfunktion zur Zugangsbeschränkung innerhalb des Funknetzes, die externe Steuerung über den Power Connect erzwingen kann.

Für die programmgesteuerte Ansteuerung stellt der Hersteller eine Python-Bibliothek bereit. In der Community wurde das Verhalten des Funkprotokolls darüber hinaus durch experimentelle Analysen („trial and error“) nachvollzogen und dokumentiert [[mikrocontroller2024gravitraxfunkfrequenz](#)]. Diese Arbeit nutzt eine C-Referenzimplementierung [[github2025alramlechnerGTProtocol](#)] als technische Grundlage, um die Struktur der Funkpakete (Protokollnachrichten) und die daraus ableitbaren Semantiken nachvollziehbar zu beschreiben.



Abbildung 2.4: Aufreihung von POWER-Elementen (z.B. Starter, Switch, Power Connect) mit Funkfähigkeit

2.1.3 Systemkomponenten und Funkfähigkeit

Die POWER-Produktlinie umfasst unterschiedliche Bauarten, deren Funkfähigkeit voneinander abweicht. Der Power Connect integriert einen Funkbaustein der XN297L-Familie (im Folgenden vereinfachend: „XN297L“), der die 2,4 GHz-Kommunikation mit den POWER-Elementen realisiert. Andere POWER-Elemente (mit Ausnahme der Lichtsteine) besitzen ebenfalls einen 2,4 GHz-Funktransceiver; die genaue Typbezeichnung kann je nach Bausteinvariante variieren und ist für die Protokollbeschreibung in dieser Arbeit nicht wesentlich.

Die Lichtsteine sind als autarke Bausteine konzipiert und verfügen über keine Funkanbindung. Die Bedienung erfolgt lokal am Baustein (über einen Taster), sodass sie nicht auf Funksignale anderer Steine reagieren. Vereinzelt finden sich Verpackungen mit dem Hinweis auf „2,4 GHz“; nach vorliegenden Informationen handelt es sich dabei um einen Fehldruck und nicht um ein Indiz für Funkfähigkeit.

2.1.4 Funkprotokoll: Nachrichtenformat und Semantik

Die untersuchten Funkpakete verwenden ein kompaktes, festes Nutzdatenformat mit 6 Byte (Payload). In der C-Implementierung wird dieses Format als „Payload“ aufgefasst und sowohl beim Senden als auch beim Empfangen unverändert verarbeitet. Die Felder lassen sich wie folgt interpretieren:

Byte	Bezeichnung	Bedeutung
1	Kanal-/Farbcodierung	Kennzeichnet den logischen Farbkanal (Rot/Grün/Blau). In der Referenzimplementierung ergibt sich der Wert als Basisoffset (0x13) plus Farbindex.
2	Senderpräfix	Identifiziert den sendenden Gerätetyp (z. B. Starter, Finish, Remote, Bridge/Connect).
3	Status-/Zielcode	Kodiert die Bedeutung des Ereignisses bzw. den adressierten Empfängertyp (z. B. Broadcast, Starter, Switch) sowie spezielle Steuerkommandos (z. B. Aktivieren/Deaktivieren eines Bridge-only-Modus).
4–5	Ereignis-ID	Pseudozufällige Kennung, die für wiederholte Aussendungen desselben Ereignisses konstant bleibt und zur Duplikaterkennung genutzt werden kann.
6	Prüfsumme	Einfache Integritätsprüfung: Summe der Bytes 1–5 modulo 256.

Tabelle 2.1: Payload-Struktur der 2,4 GHz-Funkpakete (abgeleitet aus der C-Referenzimplementierung)

Duplikaterkennung und Ereignisbehandlung Beim Empfangen wird die Prüfsumme neu berechnet und gegen das Prüfsummenbyte verglichen. Pakete mit abweichender Prüfsumme werden verworfen. Zur Vermeidung mehrfacher Reaktionen auf wiederholt gesendete Ereignisse wird zusätzlich eine einfache Duplikaterkennung eingesetzt: Stimmen Ereignis-ID und Prüfsumme mit dem zuletzt verarbeiteten Paket überein, wird dieses verworfen.

Sendeverhalten und Kanalsprungverfahren Für eine erhöhte Robustheit gegen Paketverlust werden Ereignisse mehrfach ausgesendet. Die Referenzimplementierung sendet ein identisches 6-Byte-Paket mehrfach hintereinander und wechselt dabei zwischen zwei physischen Funkkanälen pro Farbkanal. Für Rot, Grün und Blau sind jeweils zwei RF-Kanäle hinterlegt (primär und alternativ); das Umschalten erfolgt deterministisch über die Wiederholungsnummer. Zwischen zwei Aussendungen ist eine kurze Wartezeit (im Millisekundenbereich) implementiert.

Adressierung Neben der Farb-/Kanalcodierung wird in der Referenzimplementierung pro Farbkanal eine eigene 5-Byte-Funkadresse verwendet. Damit entsteht eine logische Trennung der drei Farbkanäle, die sowohl auf Adressebene als auch auf Kanälebene abgebildet wird. In Kombination mit dem Status-/Zielcode lassen sich Broadcast-Nachrichten (an alle Empfänger) sowie gerichtet interpretierbare Ereignisse für bestimmte Bausteinklassen (z. B. „Starter“ oder „Switch“) unterscheiden. Da Funkpakete

physikalisch nicht punkt-zu-punkt zugestellt werden, erfolgt die Selektion auf Empfängerseite: Steine werfen empfangene Pakete, wenn sie nicht zum konfigurierten Farbkanal passen bzw. wenn der Status-/Zielcode nicht zu ihrer Rolle im System passt.

Kodierung von Sender- und Statusfeldern (Auszug) Die in Byte 2 und Byte 3 genutzten Kodierungen sind in der Referenzimplementierung als Konstanten hinterlegt. ?? fasst die Senderpräfixe zusammen; ?? zeigt exemplarisch ausgewählte Status-/Zielcodes, die in dieser Arbeit für die weitere Analyse relevant sind.

Wert (hex)	Sender (Interpretation)
0x02	Finish-Element
0x04	Starter-Element
0x05	Remote/externes Bedienelement
0x06	Bridge/Connect (Power Connect)

Tabelle 2.2: Senderpräfixe (Byte 2) in der Referenzimplementierung

Wert (hex)	Name	Bedeutung (Interpretation)
0x00	ALL	Broadcast bzw. allgemeines Ereignis
0x01	STARTER	Ereignis/Adressierung mit Bezug zum Starter
0x02	SWITCH	Ereignis/Adressierung mit Bezug zu Switch-Elementen
0xC8	UNLOCK	Steuerkommando: Deaktivieren eines restriktiven Empfangsmodus (Bridge-only)
0xC9	LOCK	Steuerkommando: Aktivieren eines restriktiven Empfangsmodus (Bridge-only)
0xCA	STARTER_PRESS	Ereignis: Starter-Tasterbetätigung

Tabelle 2.3: Ausgewählte Status-/Zielcodes (Byte 3) in der Referenzimplementierung

2.2 Asynchrone Programmierung in Python

2.2.1 Allgemeine Motivation

Die zunehmende Bedeutung verteilter Systeme und netzwerkbasierter Anwendungen hat die Anforderungen an effiziente Nebenläufigkeitsmodelle erheblich gesteigert. Klassische Ansätze wie Multithreading oder Multiprocessing stoßen insbesondere bei I/O-gebundenen Operationen an Grenzen hinsichtlich Skalierbarkeit und Ressourcenverbrauch: Threads erzeugen zusätzliche Systemlast durch Kontextwechsel und erfordern komplexe Synchronisationsmechanismen, während Prozesse noch ressourcenintensiver

sind. Vor diesem Hintergrund hat sich die asynchrone Programmierung als effiziente Alternative etabliert, insbesondere in der Programmiersprache Python durch die Einführung der Bibliothek `asyncio` [hattingh2020].

2.2.2 Spezifische Relevanz für Hardware-Steuerung

Besondere Bedeutung gewinnt die asynchrone Programmierung bei der Steuerung verteilter Hardware-Systeme wie GraviTrax POWER, wo die Verwaltung paralleler Kommunikationskanäle und zeitabhängiger Ereignisse zentral ist. Bei der gleichzeitigen Kommunikation mit dem Power Connect-Gateway über Bluetooth Low Energy (BLE) und dem Empfang von asynchronen Funkpaketen von den POWER-Elementen (vgl. Abschnitt ??) müssen klassische Multithreading-Ansätze zu Problemen bei der Synchronisation und Reproduzierbarkeit führen. Asyncio bietet hier eine bessere Alternative durch sein kooperatives Scheduling-Modell.

2.2.3 Grundkonzepte der asynchronen Programmierung

Asynchrone Programmierung basiert auf dem Prinzip der nicht-blockierenden Ausführung. Im Gegensatz zur synchronen Programmierung, bei der Operationen sequenziell abgearbeitet werden, erlaubt dieses Paradigma die Unterbrechung von Aufgaben während Wartezeiten, um andere Operationen auszuführen. Zum Beispiel: Während eine Anwendung auf die Rückantwort eines Web-Requests wartet, kann die CPU interimweise andere Tasks ausführen, statt blockierend zu warten.

Das zentrale Konzept ist der *Event Loop*, welcher als Scheduler fungiert und die Ausführung von sogenannten *Coroutines* organisiert. Coroutines sind spezielle Funktionen, die mittels `async def` definiert werden und durch das Schlüsselwort `await` explizit unterbrochen werden können [fowler2022]. Dieses kooperative Modell unterscheidet sich grundlegend von präemptiver Nebenläufigkeit, da der Kontrollfluss bewusst durch den Entwickler gesteuert wird. Bei GraviTrax bedeutet dies beispielsweise, dass eine Coroutine auf den Empfang eines BLE-Kommandos wartet und ihre Ausführung unterbricht (`await`), bis das Kommando eintrifft, während andere Coroutines Funkpakete von POWER-Elementen verarbeiten können.

2.2.4 Architektur und Komponenten von asyncio

Die Python-Bibliothek `asyncio` implementiert mehrere zentrale Abstraktionen zur Verwaltung asynchroner Operationen:

Event Loop Koordiniert die Ausführung aller Tasks und verwaltet ein Warteschlangenbasiertes Scheduling-System.

Coroutine Pausierbare Funktionseinheit, die bei Erreichen einer `await`-Anweisung kontrolliert unterbrochen werden kann.

Task Geplante Coroutine innerhalb des Event Loops, die als eigenständige Ausführungseinheit fungiert.

Future Repräsentation eines zukünftigen Ergebnisses, das zu einem späteren Zeitpunkt verfügbar sein wird.

Diese Architektur ermöglicht eine effiziente Verwaltung paralleler I/O-Operationen, ohne dass zusätzliche Threads erforderlich sind. Untersuchungen zeigen, dass insbesondere bei Netzwerk- und Webanwendungen signifikante Performancegewinne erzielt werden können [swindell2022].

2.2.5 Anwendungsbereiche und Einsatzgebiete

Asynchrone Programmierung wird vor allem in folgenden Bereichen eingesetzt:

- **Webserver und REST-APIs:** Serverbereiche können tausende parallele Clientverbindungen effizient handhaben, ohne dass für jeden Client ein separater Thread notwendig ist.
- **Microservice-Architekturen und verteilte Systeme:** Mehrere Microservices können untereinander asynchron kommunizieren und auf gegenseitige Antworten warten, ohne Ressourcen zu verschwenden.
- **Echtzeit-Datenverarbeitung und Streamverarbeitung:** Kontinuierliche Datenströme (z. B. von Sensoren oder Netzwerkschnittstellen) können mit minimaler Latenz verarbeitet werden.
- **Simulation verteilter Systeme und Multi-Agenten-Szenarien:** Exemplarisch demonstrieren Avhad et al. [avhad2024], dass AsyncIO-basierte Systeme in Multi-Agenten-Simulationen eine effiziente Koordination paralleler Prozesse ermöglichen, indem sie autonome Agenten gleichzeitig ihre Aktionen koordinieren lassen.
- **Steuerung eingebetteter Hardware-Systeme und IoT-Geräte:** Bei Gravi-Trax POWER müssen mehrere I/O-intensive Aufgaben gleichzeitig verwaltet werden: (i) die BLE-basierte Kommunikation mit dem Power Connect-Gateway, (ii) der Empfang und die Verarbeitung von asynchronen Funkpaketen der POWER-Elemente (vgl. Abschnitt ??), sowie (iii) die zeitgerechte Ausführung von Steuererroutinen basierend auf Ereignissen aus Trigger-Elementen.

Die letzte Kategorie ist insbesondere für diese Arbeit zentral. In modernen Low-Latency-Architekturen wird der Event-Loop-Mechanismus gezielt zur Reduktion von Verzögerungen eingesetzt [umer2025], was für die reaktive Steuerung von Marmelbahn-Elementen essentiell ist, um Verzögerungen zwischen Trigger-Ereignis und Akteur-Reaktion zu minimieren.

2.2.6 Vorteile und Herausforderungen

Vorteile Ein wesentlicher Vorteil der asynchronen Programmierung liegt in der effizienten Ressourcennutzung, da keine zusätzlichen Threads erzeugt werden müssen. Dies reduziert Kontextwechsel und Speicherverbrauch erheblich [ziade2017], wodurch eine höhere Skalierbarkeit bei weniger Systemressourcen erreicht wird. Für GraviTrax bedeutet dies konkret: Eine Python-Anwendung kann parallel mit einem Power Connect kommunizieren, Funkpakete von verschiedenen POWER-Elementen verarbeiten und komplexe Steuerlogiken ausführen, ohne dass mehrere Threads verwaltet werden müssen. Zudem bietet das kooperative Modell bessere Vorhersagbarkeit des Programmablaufs, da Kontextwechsel nur an expliziten `await`-Punkten stattfinden.

Herausforderungen Jedoch bringt das Paradigma auch Herausforderungen mit sich. Die explizite Steuerung des Kontrollflusses durch `await`-Anweisungen erhöht die Komplexität des Codes und erschwert Debugging sowie Wartbarkeit, besonders wenn viele Coroutines parallel aktiv sind. Zudem ist die Eignung für CPU-intensive Aufgaben eingeschränkt, da hier weiterhin parallele Ausführungseinheiten (Multithreading oder Multiprocessing) erforderlich sind [hattingh2020]. Bei komplexeren GraviTrax-Szenarien, die aufwendige Berechnungen zur Steuerlogik-Optimierung erfordern, müsste daher eine Hybridlösung aus AsyncIO und Multiprocessing erwogen werden.

2.2.7 Zusammenfassung und Relevanz für diese Arbeit

Allgemeine Bedeutung Zusammenfassend stellt die asynchrone Programmierung in Python ein leistungsfähiges Modell zur Umsetzung nebenläufiger Systeme dar, insbesondere im Kontext I/O-intensiver Anwendungen wie der Steuerung verteilter Hardware-Systeme. Die Bibliothek `asyncio` bietet hierfür eine strukturierte und flexible Grundlage mit wohl-definierten Abstraktionen und klaren Kontrollfluss-Semantiken. Trotz ihrer Vorteile erfordert die effektive Nutzung ein fundiertes Verständnis der zugrunde liegenden Konzepte, insbesondere des Event-Loop-Mechanismus und der kooperativen Nebenläufigkeit.

Spezifische Relevanz für GraviTrax POWER-Steuerung Für die vorliegende Arbeit ist diese Kenntnis essenziell, da die Steuerung von GraviTrax POWER-Elementen über den Power Connect-Gateway die gleichzeitige Verwaltung mehrerer asynchroner Kommunikationsströme erfordert: Während BLE-Befehle an das Gateway übertragen werden, müssen parallel eingehende Funkpakete von Trigger-Elementen verarbeitet und korrespondierende Steuerroutrinen ausgelöst werden. Das kooperative Scheduling-Modell von `asyncio` bietet hier eine ideale Balance zwischen Einfachheit (gegenüber explizitem Multithreading) und Effizienz (gegenüber blockierenden I/O-Operationen) und ermöglicht damit die Implementierung einer reaktiven, ereignisgesteuerten Steuerlogik für komplexe Marmelbahn-Konfigurationen. Dies wird in den folgenden Kapiteln bei der Beschreibung der Implementierung der Steuersoftware zentral sein.

2.3 Bluetooth Low Energy

Bluetooth Low Energy (BLE) stellt eine energieeffiziente Erweiterung des klassischen Bluetooth-Standards dar und wurde insbesondere für Anwendungen mit geringem Energieverbrauch und intermittierender Datenübertragung konzipiert. Charakteristisch für BLE sind kurze Übertragungsintervalle, ein reduzierter Protokoll-Overhead sowie Betriebsmodi wie Advertising und Connection Events, die eine signifikante Minimierung des Energieverbrauchs ermöglichen (Shrestha, 2016; Koulouras et al., 2025). Im Vergleich zu klassischen Bluetooth-Systemen erreicht BLE dadurch eine deutlich verlängerte Batterielebensdauer, was insbesondere für eingebettete Systeme und IoT-Anwendungen von zentraler Bedeutung ist (Ensworth Reynolds, 2017). Die Architektur basiert auf dem Generic Attribute Profile (GATT) sowie dem Generic Access Profile (GAP), welche die Interoperabilität zwischen Geräten sicherstellen und eine flexible Kommunikation zwischen Sensoren, Aktoren und Steuergeräten ermöglichen (Aftab, 2017).

Neben klassischen Einsatzgebieten wie Wearables, Smart-Home-Systemen und industriellen IoT-Lösungen findet BLE zunehmend Anwendung in interaktiven Spielsystemen. Ein Beispiel hierfür ist das modulare Kugelbahnsystem GraviTrax, bei dem BLE zur drahtlosen Kommunikation zwischen dem Power Connect Stein und dem Anwendergerät (z. B. Smartphone, Laptop)

einzelnen Bausteinen eingesetzt wird. Durch integrierte Sensoren und Aktoren können Module Zustände erfassen (z. B. das Passieren einer Kugel) und entsprechende Signale an andere Komponenten senden, wodurch sich dynamische und synchronisierte Abläufe realisieren lassen. BLE ermöglicht hierbei eine energieeffiziente, latenzarme und robuste Vernetzung der Spielmodule, ohne dass eine kabelgebundene Infrastruktur erforderlich ist. Diese Eigenschaften sind besonders relevant für batteriebetriebene

Spielkomponenten, da sie eine lange Nutzungsdauer bei gleichzeitig hoher Interaktivität gewährleisten. Somit zeigt sich, dass BLE nicht nur im industriellen und wissenschaftlichen Kontext, sondern auch im Bereich intelligenter Spielzeuge eine zentrale Rolle bei der Realisierung vernetzter Systeme einnimmt (Byrne, 2023; Aftab, 2017).

3 Library-Analyse und Veränderungen

3.1 Zielsetzung und Einordnung

Die in dieser Arbeit verwendete Python-Bibliothek „GraviTrax Connect Python Library“ stellt die zentrale Softwareschicht zwischen einer externen Steueranwendung und dem GraviTrax-POWER-System dar [[github2023gravitraxconnectlibrary](#)]. Ihre praktische Relevanz ergibt sich aus der Rolle als BLE-Gateway: Einerseits muss sie eine stabile Verbindung zur Bridge aufbauen und verwalten, andererseits eingehende und ausgehende Signale in einer Form bereitstellen, die in anwendungsnahen Steuerprogrammen mit vertretbarem Aufwand nutzbar ist.

Aus wissenschaftlicher Perspektive ist die Bibliothek nicht nur ein Hilfsmittel, sondern ein Untersuchungsgegenstand. Die Qualität der späteren Murmelbahnsteuerungen hängt unmittelbar von ihrer Robustheit im asynchronen Laufzeitverhalten, der Konsistenz ihrer API sowie der Korrektheit der Signalauswertung ab. Vor diesem Hintergrund verfolgt das Kapitel drei Ziele: Erstens wird die Bibliothek in Aufbau und Funktion analysiert, zweitens wird ihre interne Architektur auf Schichten- und Ablaufebene beschrieben, und drittens werden die im Rahmen dieser Arbeit umgesetzten Änderungen einschließlich ihrer Motivation und ihres Nutzens begründet.

3.2 Struktur und Architektur der Bibliothek

Die Bibliothek ist modular aufgebaut und trennt Konfigurationswissen, Kommunikationslogik und Anwendungscode. Diese Trennung ist insbesondere für wissenschaftlich orientierte Weiterentwicklung relevant, da dadurch Änderungen an Protokollparametern, Laufzeitverhalten und Schnittstellenverträgen isoliert untersucht werden können.

3.2.1 Modulaufbau

Die Kernfunktionalität liegt im Paket `gravitraxconnect` mit zwei zentralen Modulen:

- `gravitrax_bridge.py`: Implementiert die komplette BLE-Kommunikation, Signalverarbeitung und Sende-/Empfangslogik.
- `gravitrax_constants.py`: Definiert Protokollkonstanten (UUIDs, Farbkanäle, Statuscodes, Steintypen, Batteriemapping).

Ergänzt wird dies durch:

- Anwendungsbeispiele (Ordner `examples`),
- mehrere Demo-Applikationen (CLI, GUI, GPIO, Timer, Reaction Game),
- testbezogene Skripte (hardwareabhängige Funktionstests und neue hardwareunabhängige Unit-Tests).

3.2.2 Architekturebenen

Auf konzeptioneller Ebene lässt sich die Bibliothek in vier Schichten gliedern:

1. **Konstanten- und Protokollebene:** In `gravitrax_constants.py` sind UUIDs, Statuscodes, Farbcodes und Steintypen zentral definiert.
2. **Transportebene (BLE):** Die Bibliothek kapselt die `bleak`-Funktionalität für Scanning, Verbindungsaufbau sowie GATT-Zugriffe.
3. **Protokoll- und Ablaufebene:** Hier werden Bytefolgen validiert, Signale konstruiert, Prüfsummen berechnet und Benachrichtigungen semantisch aufbereitet.
4. **Anwendungsebene:** Nutzerprogramme registrieren Callbacks und steuern die Bahn ausschließlich über die öffentliche API.

Die Schichtung folgt damit einem klaren Entkopplungsprinzip: Low-Level-BLE-Details bleiben im Bibliothekskern verborgen, während die Anwendungsebene mit semantischen Parametern (Status, Farbe, Steintyp) arbeitet. Für die in dieser Arbeit entwickelte Steuerlogik ist dies zentral, da dadurch die Experimente primär auf Fachlogik und nicht auf Transportdetails fokussieren.

3.2.3 Kernklasse Bridge als Zustandscontainer

Die Klasse `Bridge` bildet das zentrale Abstraktionsobjekt. Sie kapselt den Zustand einer BLE-Verbindung inklusive Geräteidentität, Firmware-/Hardwaredaten, Callback-Referenzen und interner Synchronisationsmechanismen.

Relevante Entwurfsmerkmale sind:

- asynchrones API-Design auf Basis von `asyncio`,
- entkoppelte Callback-Schnittstellen für Disconnect- und Notification-Ereignisse,
- Sperrmechanismen (`asyncio.Lock`) für kritische Bereiche bei Notification-Auswertung und Nachrichten-ID-Vergabe,
- zusätzliche Utility-Funktionen für Scans, Prüfsummen und Logging.

Die folgende Initialisierung zeigt exemplarisch, dass Verbindungszustand, Callback-Registrierung und Synchronisationsobjekte in einem einzigen Objekt zusammengeführt werden.

Listing 3.1: Auszug: Zustandsinitialisierung der Bridge-Klasse

```
class Bridge:
    def __init__(self):
        self.client = None
        self.name = None
        self.firmware = None
        self.hardware = None
        self.id = 0
        self.dc_callback = None
        self.try_reconnect = False
        self.restart_notifications = True
        self.noti_callback = None
        self.__noti_lock = asyncio.Lock()
        self.__send_lock = asyncio.Lock()
        self.__address = None
        self.__next_send_id = 0
        self.__user_disconnected = False
        self.__last_notifications = []
```

Methodisch bedeutet dies, dass die Bibliothek zustandsorientiert implementiert ist. Der Vorteil liegt in einer klaren Objektidentität für eine Bridge-Verbindung; der Nachteil ist eine stärkere Kopplung von Lebenszyklus und Laufzeitdaten innerhalb derselben Klasse. Für die Zielanwendung dieser Arbeit überwiegt der Vorteil, weil die Steuerung einer einzelnen aktiven Bridge der dominierende Anwendungsfall ist.

3.3 Funktionale Analyse

3.3.1 Verbindungsaufbau und Lebenszyklus

Der Verbindungsaufbau erfolgt über `connect()`, wahlweise per Gerätenamen oder MAC-Adresse. Intern wird zunächst ein Device-Discovery-Schritt über den BLE-Scanner

durchgeführt und anschließend ein `BleakClient` initialisiert. Nach erfolgreicher Verbindung werden Geräteinformationen (Firmware, Hardware) abgefragt.

Die Trennung zwischen `connect()`, `is_connected()` und `disconnect()` ist klar gehalten und erlaubt eine robuste Ablaufsteuerung in asynchronen Anwendungen. Über `dc_callback` und optionale Reconnect-Logik kann auf Verbindungsverluste reaktiv reagiert werden.

Der folgende Auszug zeigt den grundlegenden Ablauf des Verbindungsaufbaus.

Listing 3.2: Auszug: Verbindungsaufbau per Name oder Adresse

```
async def connect(self, name_or_addr="GravitraxConnect", by_name=True,
↪ timeout=25):
    if by_name:
        device = await BleakScanner.find_device_by_filter(
            lambda d, ad: d.name and d.name.lower() == name_or_addr.lower(),
            ↪ timeout
        )
    else:
        device = await BleakScanner.find_device_by_address(name_or_addr,
            ↪ timeout)

    if not isinstance(device, BLEDevice):
        return False

    self.client = BleakClient(device,
        ↪ disconnected_callback=self.__disconnect_callback)
    await self.client.connect(timeout=timeout)
    self.name = device.name
    self.__address = device.address
    await self.request_bridge_info()
    return True
```

Aus Architektursicht ist hervorzuheben, dass Discovery und Verbindung nicht in separate Komponenten ausgelagert sind. Die Bibliothek priorisiert hier ein kompaktes API-Design gegenüber strikter Trennung. Für Forschungs- und Prototypingkontexte ist dies pragmatisch, da die Nutzung mit wenigen Aufrufen möglich bleibt.

3.3.2 Senden von Steuerinformationen

Für ausgehende Kommunikation existieren drei Ebenen:

- `send_bytes()`: Low-Level-Senden beliebiger Bytefolgen,
- `send_signal()`: Erzeugung protokollkonformer GraviTrax-Signale,
- `send_periodic()`: wiederholtes Senden mit zeitlichem Abstand.

Ein Sendesignal wird als 7-Byte-Struktur aufgebaut (Header, Steintyp, Status, Reserved, Message-ID, Checksumme, Farbkanal). Die Message-ID wird sequenziell oder zufällig vergeben; zusätzlich können Mehrfachsendungen („resends“) zur Reduktion von Paketverlusten genutzt werden.

Die Signalkonstruktion erfolgt explizit als Byte-Serialisierung. Der entsprechende Codeausschnitt macht sichtbar, dass die Message-ID-Vergabe als kritischer Abschnitt synchronisiert wird.

Listing 3.3: Auszug: Konstruktion und Versand eines GraviTrax-Signals

```
async def send_signal(self, status, color_channel, random_id=False,
    ↪ stone=STONE_BRIDGE):
    reserved = random.randrange(0, 255)
    async with self.__send_lock:
        if random_id:
            message_id = random.randrange(0, 255)
        else:
            message_id = self.__next_send_id
        self.__next_send_id = (self.__next_send_id + 1) % 256

    checksum = (MSG_DEFAULT_HEADER + stone + status + reserved + message_id)
    ↪ % 256
    await self.send_bytes(bytes([
        MSG_DEFAULT_HEADER, stone, status, reserved, message_id, checksum,
        ↪ color_channel
    ]))
```

Diese Implementierung illustriert den Entwurfsansatz der Bibliothek: Protokollwissen bleibt intern, die Anwendung übergibt semantische Parameter. Gleichzeitig ermöglicht die API mit `send_bytes()` eine niedrigere Zugriffsebene für Experimente, was für explorative Arbeiten mit proprietären Protokollen vorteilhaft ist.

3.3.3 Empfang und Notification-Verarbeitung

Eingehende BLE-Notifications werden über `notification_enable()` aktiviert und intern in `__notification_handler()` verarbeitet. Die Implementierung extrahiert mögliche GraviTrax-Signale mittels Mustererkennung, dekodiert die Felder und übergibt sie an den nutzerdefinierten Callback.

Wesentliche Schutzmechanismen sind:

- Prüfsummenvalidierung zur Erkennung fehlerhafter Pakete,
- Duplikaterkennung über einen Verlauf zuletzt empfangener Signale,

- Synchronisation über einen Lock, um konkurrierende Callback-Ausführungen konsistent zu halten.

Der nächste Ausschnitt zeigt den Kern der eingehenden Signalauswertung, insbesondere die Prüfsummenprüfung und das Weiterreichen nur valider Daten.

Listing 3.4: Auszug: Validierung eingehender Notifications

```
async def __notification_handler(self, characteristic, data):
    message = " ".join("{:02x}".format(x) for x in data)
    recv_signals = [m.group() for m in
        ↪ re.finditer(r"13(\s[0-9a-fA-F]{2}){6}", message)]

    async with self.__noti_lock:
        for recv_signal in recv_signals:
            if recv_signal in self.__last_notifications:
                continue
            self.__last_notifications.append(recv_signal)

            signal = [
                int(recv_signal[0:2], 16), int(recv_signal[3:5], 16),
                int(recv_signal[6:8], 16), int(recv_signal[9:11], 16),
                int(recv_signal[12:14], 16), int(recv_signal[15:17], 16),
                int(recv_signal[18:20], 16),
            ]

            if signal[5] != calc_checksum(signal, for_received=True):
                continue

            await self.noti_callback(self, Header=signal[0], Stone=signal[1],
                                    Status=signal[2], Id=signal[4],
                                    ↪ Color=signal[6])
```

Die Bibliothek folgt damit einem klaren „parse-validate-dispatch“-Muster. Dieses Muster ist für echtzeitnahe Steuerung zweckmäßig, da fehlerhafte Pakete früh verworfen und gültige Ereignisse mit geringer zusätzlicher Latenz an die Anwendung weitergegeben werden.

3.3.4 Scan- und Hilfsfunktionen

Die Funktion `scan_bridges()` unterstützt die Discovery verfügbarer Bridges mit Filteroptionen, Timeout und optionalem frühzeitigem Abbruch. Die Utility-Funktionen `calc_checksum()`, `add_checksum()` und Logging-Helfer ergänzen die API um wiederverwendbare Basisbausteine.

Gerade `scan_bridges()` ist architektonisch relevant, weil diese Funktion den Übergang vom ungebundenen Systemzustand (keine Verbindung) in den adressierbaren Be-

triebszustand vorbereitet. Die in dieser Arbeit vorgenommene Überarbeitung verbessert hier die deterministische Laufzeitsteuerung (genauere Timeout-Behandlung und sauberes frühes Stoppen).

3.4 Identifizierte Schwachstellen vor der Überarbeitung

Bei der Analyse der Ausgangsbasis zeigten sich mehrere praxisrelevante Punkte:

1. In `connect()` lieferte ein Timeout in einem Pfad keinen expliziten booleschen Rückgabewert, obwohl die Methode als `bool` konzipiert ist.
2. Fehlerhafte Notifications (ungültige Prüfsumme) wurden protokolliert, jedoch im selben Ablauf weiterhin an den Nutzer-Callback weitergereicht.
3. Die manuelle Lock-Steuerung im Sendepfad war unnötig fehleranfällig (Acquire/Release ohne Kontextmanager).
4. `scan_bridges()` verlor durch Integer-Cast Timeout-Genauigkeit und setzte den Modus `stop_on_hit` nicht sauber um.
5. Logging-Handler konnten bei wiederholtem Import mehrfach angelegt werden.
6. Dokumentation und Metadaten waren in Teilen inkonsistent (Version, Dokumentationslink, Beispiele, Mindest-Python-Version).

Diese Schwachstellen sind nicht unabhängig voneinander. Insbesondere inkonsistente Rückgabewerte, unzureichende Datenvalidierung und unpräzises Scan-Verhalten verstärken sich in asynchronen Steuerprogrammen gegenseitig: Fehler werden dort typischerweise nicht sofort als Ausnahme sichtbar, sondern manifestieren sich als intermittierende Laufzeitartefakte (fehlende Triggerreaktionen, nicht reproduzierbare Zustände, uneinheitliches Anwendungsverhalten). Die Überarbeitung zielte daher explizit auf Determinismus und beobachtbare Konsistenz.

3.5 Durchgeführte Änderungen und Begründung

Im Rahmen dieser Studienarbeit wurden die oben genannten Punkte systematisch adressiert.

3.5.1 Korrekturen in der Kernlogik

1) **Eindeutige Rückgabesemantik in `connect()`.** Bei einem Verbindungstimeout wird nun explizit `False` zurückgegeben. Dies stellt sicher, dass Aufruerverhalten (z. B. bedingte Ablaufsteuerungen) deterministisch bleibt.

2) **Verwerfen fehlerhafter Notifications.** Nach erkannter Prüfsummenabweichung erfolgt nun ein `continue` im Handler. Damit werden beschädigte Pakete konsequent verworfen und gelangen nicht mehr in die Nutzerlogik. Die Änderung verbessert die Datenintegrität der Ereignisverarbeitung.

3) **Sicherere Synchronisation im Sendepfad.** Die manuelle Sperrenverwaltung wurde durch `async with` ersetzt. Dadurch wird die Freigabe der Sperre auch bei Ausnahmen automatisch sichergestellt; das Risiko von Deadlocks sinkt.

4) **Robusteres Scan-Verhalten.** `scan_bridges()` nutzt Timeouts nun als `float` ohne Abschneiden und implementiert `stop_on_hit` über ein Ereignisobjekt mit `wait_for()`. Gleichzeitig wird der Scanner in einem `finally`-Block stets korrekt gestoppt. Dies verbessert sowohl Präzision als auch Ressourcenbereinigung.

Listing 3.5: Auszug: Überarbeitetes Scan-Verhalten mit frühem Abbruch

```
async def scan_bridges(name="GravitraxConnect", timeout=10.0,
    ↪ stop_on_hit=False):
    timeout = float(timeout)
    address_list = []
    stop_event = asyncio.Event()

    def discover_callback(device, advertisement_data):
        if (device.name == name or name == "") and device.address not in
            ↪ address_list:
            address_list.append(device.address)
            if stop_on_hit:
                stop_event.set()

    scanner = BleakScanner(discover_callback, "")
    await scanner.start()
    try:
        if stop_on_hit:
            try:
                await asyncio.wait_for(stop_event.wait(), timeout=timeout)
            except TimeoutError:
                pass
        else:
            await asyncio.sleep(timeout)
    finally:
        await scanner.stop()
    return address_list
```

5) Korrektur von Fehlermeldungstexten. In mehreren `TypeError`-Pfaden wurden String-Formatierungen und Meldungstexte berichtigt, damit Diagnosen im Fehlerfall eindeutig sind.

6) Vermeidung mehrfacher Logger-Handler. Die Logger-Initialisierung prüft nun, ob ein benannter Stream-Handler bereits existiert. Damit werden doppelte Logausgaben in interaktiven oder mehrfach initialisierten Laufumgebungen vermieden.

3.5.2 Dokumentation und Paketmetadaten

Zusätzlich zur Kernlogik wurden projektbegleitende Artefakte angepasst:

- Versionsanhebung auf 0.3.4 in `pyproject.toml`,
- Eintrag eines validen Dokumentationslinks,
- inhaltliche Korrektur der README (Mindestversion Python 3.10, präzisere Formulierungen),
- Ergänzung eines Abschnitts zur Testausführung,
- Korrektur mehrerer Codebeispiele in der Bibliotheksdokumentation (insbesondere hinsichtlich Rückgabeverhalten der asynchronen Sendefunktionen).

3.5.3 Qualitätssicherung durch neue Unit-Tests

Zur Absicherung der Änderungen wurden hardwareunabhängige Unit-Tests ergänzt. Diese decken insbesondere Regressionen in den kritischen Stellen ab:

- korrektes `False` bei `Connect-Timeout`,
- Verwerfen falscher Prüfsummen im `Notification-Handler`,
- präzise `Float-Timeout`-Verwendung beim `Scan`,
- funktionales frühzeitiges Stoppen bei `stop_on_hit=True`.

Durch den Einsatz von Mocks (u. a. für Scanner- und Client-Verhalten) sind diese Tests ohne physische Bridge ausführbar und verbessern damit die Reproduzierbarkeit im Entwicklungsprozess.

Ein Beispielttest zur Prüfsummenvalidierung ist nachfolgend dargestellt.

Auf methodischer Ebene ist bedeutsam, dass diese Tests nicht die reale BLE-Hardware ersetzen sollen, sondern eine komplementäre Prüfebene bereitstellen. Hardwaretests bleiben für End-to-End-Validierung notwendig; Unit-Tests sichern dagegen kontrolliert einzelne Invarianten ab.

Listing 3.6: Auszug: Unit-Test zum Verwerfen fehlerhafter Notifications

```
async def test_notification_handler_drops_wrong_checksum(self):
    bridge = gb.Bridge()
    received = 0

    async def callback(_bridge, **_signal):
        nonlocal received
        received += 1

    bridge.noti_callback = callback
    bad_signal = bytearray([0x13, 0x01, 0x02, 0x03, 0x04, 0x00, 0x01])
    await bridge._Bridge__notification_handler(None, bad_signal)

    self.assertEqual(received, 0)
```

3.6 Bewertung der vorgenommenen Änderungen

Die Änderungen adressieren keine grundlegende Neugestaltung der Bibliothek, sondern gezielte Verbesserungen in Robustheit, Korrektheit und Wartbarkeit. Insbesondere die Konsolidierung der Rückgabesemantik, die korrekte Behandlung beschädigter Notifications und das verbesserte Scan-Handling wirken sich direkt auf die Zuverlässigkeit von anwendungsseitigen Steuerprogrammen aus.

Aus methodischer Sicht ist hervorzuheben, dass die Erweiterung um automatisierte Unit-Tests die Bibliothek von einem primär hardwarezentrierten Validierungsansatz (Funktionstests mit realer Bridge) um eine softwareseitig reproduzierbare Qualitätssicherung ergänzt. Damit wird die Weiterentwicklung der Bibliothek auch ohne permanente Hardwareverfügbarkeit erleichtert.

Architektonisch bleibt die Bibliothek bewusst schlank und anwendungsnah. Die vorgenommenen Änderungen erhöhen die strukturelle Qualität, ohne das API-Paradigma zu brechen. Dies ist für bestehende Nutzercodes besonders vorteilhaft, weil Kompatibilität erhalten bleibt, während das Laufzeitverhalten berechenbarer wird.

Für die in dieser Arbeit entwickelten Marmelbahnprogramme ergeben sich daraus zwei direkte Effekte: Zum einen sinkt die Wahrscheinlichkeit schwer reproduzierbarer Kommunikationsartefakte, zum anderen steigt die Aussagekraft experimenteller Ergebnisse, weil beobachtete Effekte eher auf die Fachlogik als auf Bibliotheksinstabilitäten zurückgeführt werden können.

3.7 Fazit des Kapitels

Die GraviTrax-Connect-Python-Library stellt eine funktional geeignete Grundlage für die in dieser Arbeit realisierten Marmelbahnsteuerungen dar. Die durchgeführten Anpassungen verbessern die Verlässlichkeit der BLE-Kommunikation, reduzieren potenzielle Fehlerquellen im asynchronen Ablauf und erhöhen die Nachvollziehbarkeit durch konsistente Dokumentation und Tests.

Die überarbeitete Bibliotheksbasis bildet damit eine tragfähige Grundlage für die in den folgenden Kapiteln dargestellten anwendungsnahen Steuerlogiken und Versuchsaufbauten.

4 Murmelbahnaufbauten

4.1 Aufbau 1: Marbels2Binary

Der Murmelbahnaufbau *Marbels2Binary* verfolgt das Ziel, die Anzahl der durch den Nutzer via dem Startblock freigegebenen Murmeln als 4-Bit-Binärzahl darzustellen. In einem Zyklus werden dabei bis zu 15 Startvorgänge erfasst (entsprechend dem Wertebereich 0 bis 15), die resultierende Binärdarstellung wird jedoch nicht über vier gleichzeitig sichtbare Murmeln umgesetzt, sondern über die Auswahl von bis zu vier “repräsentativen” Murmeln: Aus der gezählten Gesamtanzahl werden genau diejenigen Murmeln in eine Binär-Sequenz geführt, die den gesetzten Bits (8, 4, 2, 1) entsprechen. Die verbleibenden Murmeln werden verworfen. Damit entsteht eine direkte, physische Repräsentation des Zählergebnisses durch die Anwesenheit der Murmeln in den vier Bit-Lanes.

4.1.1 Konzept und Aufbau

Das Gesamtsystem kombiniert eine mechanische Murmelbahn mit elektronisch ansteuerbaren GraviTrax-Connect-Elementen (insbesondere Starter, Weichen/Switches, Trigger und Kanone) sowie einer softwarebasierten Steuerung. Die Steuerung übernimmt ein Python-Skript, das über die Bridge-Schnittstelle Ereignisse (Notifications) der Bahn empfängt und im Gegenzug Schaltsignale sowie Kanonenschüsse auslöst. Der Aufbau gliedert sich funktional in (i) einen Startbereich, der die Benutzereingaben (Freigabe von Murmeln) erzeugt, (ii) einen Zähl-/Bestätigungsbereich, in dem Trigger-Signale als Rückmeldung über tatsächlich passierte Murmeln dienen, sowie (iii) einen Binär-Auswahlbereich, der die Murmeln in vier definierte Lanes (Bits 8, 4, 2, 1) sortiert. Abbildungen des physischen Aufbaus sind in Abbildung X dargestellt.

Eine wesentliche Randbedingung ergibt sich aus der verwendeten Hardwaretopologie: Die rote Schaltkanal-Ansteuerung beeinflusst im Aufbau sowohl eine Weiche für den “Keep/Discard”-Ausstieg als auch die rote Weiche innerhalb der Binärselektion. Um unerwünschte Seiteneffekte zu vermeiden, arbeitet die Steuerung daher nicht mit rein phasenbasiertem Umschalten, sondern hält den Schaltzustand als Paritätszustand (“getoggelt” vs. “Default”) in Software nach und stellt vor jedem kritischen Schritt explizit

den erwarteten Zustand her.

4.1.2 Funktionsweise

Die Steuerung des Aufbaus ist ereignisgetrieben und asynchron umgesetzt. Kernidee ist, dass die Murmelbahn einerseits physische Prozesse (Rollbewegung, Schaltzeiten der Weichen) besitzt, andererseits aber über die Bridge fortlaufend digitale Notifications liefert, die als Rückmeldung für die Steuerlogik dienen. Die Software implementiert damit eine geschlossene Regelung: Die Hardware wird angesteuert (Schalten, Schießen), und das erfolgreiche Eintreten von Teilergebnissen (“Murmel wurde gezählt”, “Murmel hat den Keep-Pfad passiert”, “Murmel ist in die Binärselektion eingetreten”) wird über Triggerereignisse bestätigt. Die nachfolgend erläuterten Mechanismen sind in `figures/marbels2binary_working.py` implementiert.

Zustandsmodell und Notification-Verarbeitung

Zur Synchronisation zwischen eintreffenden Notifications und laufenden Steueraufgaben wird ein expliziter Zustand gehalten. Das Objekt *ReleaseState* dient dabei sowohl als Zähler für Starter-Auslösungen als auch als Träger von Events, die auf Trigger-Rückmeldungen warten können (*asyncio.Event*). Parallel wird in *SwitchState* die Parität der Schaltkanäle relativ zur Default-Stellung mitgeführt, um jederzeit den aktuell angenommenen Schaltzustand explizit herstellen zu können.

Die eigentliche Ereignisinterpretation erfolgt in einer Notification-Callback-Funktion. Diese ordnet eingehende Notifications anhand von Header, Stone-, Status- und Color-Feldern den jeweiligen Teilprozessen zu: Starter-Events erhöhen den Release-Zähler, Trigger-Events setzen Zeitstempel und signalisieren wartenden Tasks, dass ein erwarteter Prozessschritt erfolgt ist. Gleichzeitig kann die Callback-Logik über *state.mode* zwischen Zählen, Keep-Bestätigung und Binär-Eintritt umschalten.

Zählphase: Release Window mit Inaktivitätsende

Die Zählung der vom Nutzer freigegebenen Murmeln erfolgt nicht über eine fixe Eingabezeit, sondern über ein Inaktivitätskriterium. Sobald mindestens eine Murmel registriert wurde und für eine definierte Dauer keine weitere Starter-Auslösung erkannt wird, wird die Zählphase abgeschlossen. Dieses Vorgehen ist insbesondere für reale Bedienvorgänge geeignet, da Nutzer Murmeln nicht immer in konstanten Intervallen freigeben.

Der Abschluss der Zählphase liefert die Variable *released*. Daraus werden die Popcount-Anzahl der Keep-Murmeln sowie die Anzahl der zu verworfenden Murmeln bestimmt.

Der Popcount ist hierbei die Anzahl gesetzter Bits in der Binärdarstellung und entspricht damit genau der Anzahl Murmeln, die in die vier Bit-Lanes verteilt werden müssen.

Binär-Mapping und Zielzustand der Weichen

Die Zuordnung der Bits zu den physischen Lanes erfolgt über eine feste Topologie. Aus der gezählten Zahl wird zunächst die Menge der gesetzten Bits in absteigender Reihenfolge extrahiert. Anschließend wird für jedes Bit ein Sollzustand der drei Schaltkanäle (rot, grün, blau) berechnet, der exakt eine Lane adressiert.

Damit die Zielzustände auch unter den Bedingungen eines nicht deterministischen Hardwareverhaltens zuverlässig erreicht werden, stellt die Steuerung die Weichenstellungen nicht implizit über “Phasenwechsel” her, sondern vergleicht stets Ist- und Sollzustand und toggelt ausschließlich die Kanäle, die tatsächlich abweichen. Zusätzlich wird die Umschaltreihenfolge so gewählt, dass Seiteneffekte des roten Kanals möglichst spät (und nur falls erforderlich) auftreten.

Keep-Phase: zustandssicheres Schalten mit Trigger-Feedback und Timeouts

Die eigentliche Sortierung erfolgt während des “Keep”-Teils der Sequenz. Für jede Keep-Murmel wird (i) der Zielzustand der Binär-Weichen vorbereitet, (ii) eine Murmel über die Kanone ausgelassen und (iii) das erfolgreiche Passieren des Keep-Pfades per Trigger bestätigt. Bleibt diese Bestätigung aus, wird als Fehlmaßnahme automatisch ein Schaltkanal erneut getoggelt und der Versuch wiederholt. Die Notwendigkeit dieser Rückkopplung ergibt sich aus sporadischen Fehlstellungen einzelner Weichen.

Eine zentrale Besonderheit liegt darin, dass die rote Schaltansteuerung im Aufbau zwei Weichen beeinflusst (Ausstieg Keep/Discard und rote Weiche der Binär-Selektion). Deshalb wird der rote Kanal nicht frühzeitig auf das Bitziel geschaltet, sondern erst nachdem durch den roten Trigger bestätigt ist, dass die Murmel den Ausstieg korrekt passiert hat. Erst danach wird der rote Kanal auf den Zielzustand für die Bit-Lane finalisiert.

Nach erfolgreicher Keep-Phase werden die verbleibenden Murmeln (Discard) in einem Block über die Kanone in einen definierten Verwerfpfad geschossen. Vor dem Discard-Schritt wird dabei erneut der notwendige Ausstiegzustand hergestellt, damit keine unbestätigten “Keep”-Murmeln mehr in der Bahn befinden und die Binär-Lanes beeinflussen können.

4.1.3 Aufgetretene Probleme

Im praktischen Betrieb traten mehrere Probleme auf, die sowohl die Zuverlässigkeit des Systems als auch die Konzeption beeinflussten. Erstens wurde der sogenannte Bridge-only-Modus nicht durchgängig von allen Elementen korrekt erkannt. Insbesondere Weichen/Switches verblieben vereinzelt im Normalmodus und verarbeiteten Trigger-Signale autonom. Dadurch entstanden unbeabsichtigte Schaltzustände, die in der Folge zu inkonsistentem Routing und fehlerhaften Binärdarstellungen führten.

Zweitens zeigte mindestens ein Switch ein sporadisches Fehlverhalten beim Umschalten. Unregelmäßig blieb die Weiche in einer Ausrichtung (insbesondere in einer linksgerichteten Stellung) hängen, obwohl ein Schaltsignal gesendet wurde. Das Problem verschwand ohne erkennbares Muster nach mehreren erneuten Schaltversuchen, führte in der Zwischenzeit jedoch zu nicht deterministischem Systemverhalten. Für die Steuerung bedeutete dies, dass rein offene-loop (d. h. ohne Rückmeldung) ausgeführte Schaltsequenzen nicht ausreichen; die im Skript implementierte Strategie mit Trigger-Feedback, Timeouts und automatischem Nachschalten ist eine direkte Reaktion auf diese Beobachtung.

Drittens erwies sich in der Konzeptionsphase das Fehlen eines dedizierten “Stop-” bzw. Gate-Elements als limitierend. Für eine zuverlässige Signalauslösung wäre es vorteilhaft gewesen, Murmeln an definierten Stellen zu puffern und kontrolliert einzeln freizugeben. Ein solches Element existiert im verwendeten System nicht in der gewünschten Form. Als funktionaler Ersatz kann die elektronische Kanone genutzt werden, da sie Murmeln einzeln freigibt und mit hoher Frequenz (typischerweise etwa 250 ms pro Murmel) auslösen kann. Gleichzeitig erzeugt die Kanone jedoch einen Impuls, sodass die Anfangsgeschwindigkeit der Murmeln aktiv berücksichtigt werden muss. In der Praxis erforderte dies eine geeignete Zuführung, damit Murmeln weder zu schnell in die Kanone rollen (und auf der Gegenseite zurückrollen) noch zu langsam bzw. unter zu geringem Gefälle ankommen (und die Kanone nicht erreichen). Das mitgelieferte Anstell-Element (vgl. Abbildung X) stellte hierfür eine praktikable Lösung dar, da es einen halben Höhenblock überbrückt und damit ein Verklemmen beim Anstellen reduziert.

Viertens erwies sich der Lever nicht als geeignetes Stop-Element. Bei mehreren anstehenden Murmeln stößt der Lever beim Auslösen die gesamte Murmelreihe zurück; dabei geht mit zunehmender Anzahl an Murmeln mechanische Energie verloren. Abhängig von der Länge der Warteschlange konnte dies dazu führen, dass nachfolgende Elemente nicht mehr sicher erreicht wurden und die Murmelbahn in einem Stillstand endete.

Listing 4.1: Zustandsobjekte zur Zählung und zur Verfolgung der Schaltparität

```
class ReleaseState:
    def __init__(self, max_marbles: int):
        self.max_marbles = max_marbles
        self.count = 0
        self.last_release_ts: Optional[float] = None
        self.finished = asyncio.Event()
        self.mode = "count_releases"
        self.required_passes = 0
        self.passed_count = 0
        self.pass_event = asyncio.Event()
        self.red_trigger_event = asyncio.Event()
        self.green_trigger_event = asyncio.Event()
        self.blue_trigger_event = asyncio.Event()
        self.binary_entry_count = 0
        self.binary_entry_event = asyncio.Event()

    def add_release(self) -> None:
        if self.count >= self.max_marbles:
            return
        self.count += 1
        self.last_release_ts = time.monotonic()
        if self.count >= self.max_marbles:
            self.finished.set()

class SwitchState:
    def __init__(self):
        self.red_toggled = False
        self.green_toggled = False
        self.blue_toggled = False

    def on_toggle(self, color_channel: int) -> None:
        if color_channel == gc.COLOR_RED:
            self.red_toggled = not self.red_toggled
        elif color_channel == gc.COLOR_GREEN:
            self.green_toggled = not self.green_toggled
        elif color_channel == gc.COLOR_BLUE:
            self.blue_toggled = not self.blue_toggled
```

Listing 4.2: Auszug: Notification-Callback zur Ereignisklassifikation und Rückmeldung

```

async def notification_callback(bridge: Any, **signal) -> None:
    if (
        signal.get("Header") == gc.MSG_DEFAULT_HEADER
        and signal.get("Stone") == gc.STONE_TRIGGER
    ):
        now = time.monotonic()
        trigger_color = signal.get("Color")
        if trigger_color == pass_trigger_color:
            state.last_red_trigger_ts = now
            state.red_trigger_event.set()
        if trigger_color == binary_entry_trigger_color:
            state.last_green_trigger_ts = now
            state.green_trigger_event.set()
        if trigger_color == blue_trigger_color:
            state.last_blue_trigger_ts = now
            state.blue_trigger_event.set()

    if state.mode == "count_releases":
        if not is_user_release_signal(signal, starter_color):
            return
        state.add_release()
        return

    if state.mode == "wait_keep_passes":
        if not is_keep_path_pass_signal(signal, pass_trigger_color):
            return
        state.add_pass()
        return

    if state.mode == "wait_binary_entry":
        if not is_binary_entry_signal(signal, binary_entry_trigger_color):
            return
        state.add_binary_entry()

```

Listing 4.3: Auszug: Inaktivitätsbasierte Beendigung der Zählung

```

async def wait_for_release_window(state: ReleaseState, inactivity_seconds:
    float) -> None:
    while not state.finished.is_set():
        if state.count > 0 and state.last_release_ts is not None:
            idle_for = time.monotonic() - state.last_release_ts
            if idle_for >= inactivity_seconds:
                state.finished.set()
                return
        await asyncio.sleep(0.05)

```

Listing 4.4: Auszug: Ableitung von Keep/Discard aus der gezählten Murmelanzahl

```
released = counted
binary_repr = format(released, "b")
keep_count = released.bit_count()
discard_count = released - keep_count
```

Listing 4.5: Auszug: Bit-Extraktion und Zuordnung auf Schaltzustände (rot, grün, blau)

```
def binary_bits_desc(value: int) -> list[int]:
    return [bit for bit in (8, 4, 2, 1) if value & bit]

def target_switch_state_for_bit(bit_value: int) -> tuple[bool, bool, bool]:
    mapping = {
        8: (False, False, False),
        4: (False, False, True),
        2: (False, True, False),
        1: (True, False, False),
    }
    return mapping[bit_value]
```

Listing 4.6: Auszug: Zustandsabgleich und selektives Umschalten der Binär-Weichen

```
async def set_binary_switch_state(
    bridge: Any,
    switch_state: SwitchState,
    target_state: tuple[bool, bool, bool],
    switch_settle: float,
    resends: int,
    resend_gap: float,
) -> tuple[bool, bool, bool]:
    red_state, green_state, blue_state = switch_state.binary_tuple()
    target_red, target_green, target_blue = target_state

    # Toggle only changed channels to keep red-channel side effects as low as
    ↪ possible.
    if blue_state != target_blue:
        await send_switch_toggle(
            bridge=bridge,
            switch_state=switch_state,
            color_channel=gc.COLOR_BLUE,
            resends=resends,
            resend_gap=resend_gap,
            switch_settle=switch_settle,
        )
        blue_state = switch_state.blue_toggled

    if green_state != target_green:
        await send_switch_toggle(
            bridge=bridge,
            switch_state=switch_state,
            color_channel=gc.COLOR_GREEN,
            resends=resends,
            resend_gap=resend_gap,
            switch_settle=switch_settle,
        )
        green_state = switch_state.green_toggled

    if red_state != target_red:
        await send_switch_toggle(
            bridge=bridge,
            switch_state=switch_state,
            color_channel=gc.COLOR_RED,
            resends=resends,
            resend_gap=resend_gap,
            switch_settle=switch_settle,
        )
        red_state = switch_state.red_toggled

    return red_state, green_state, blue_state
```

Listing 4.7: Auszug: Keep-Phase mit Auto-Toggle bei fehlender Rückmeldung und späterer Finalisierung des roten Kanals

```
# Ensure marbles leave through keep path before each keep shot.
if switch_state.red_toggled != keep_exit_red_toggled:
    await send_switch_toggle(
        bridge=bridge,
        switch_state=switch_state,
        color_channel=switch_toggle_color,
        resends=resends,
        resend_gap=resend_gap,
        switch_settle=switch_settle,
    )
    auto_toggles += 1

next_pass_target = state.passed_count + 1
state.pass_event.clear()
state.red_trigger_event.clear()
state.mode = "wait_keep_passes"
shot_ts = time.monotonic()

await bridge.send_signal(
    status=gc.STATUS_CANNON,
    color_channel=gc.COLOR_RED,
    resends=resends,
    resend_gap=resend_gap,
)

try:
    await asyncio.wait_for(
        wait_for_pass_target(state, next_pass_target),
        timeout=pass_timeout,
    )
except asyncio.TimeoutError:
    await send_switch_toggle(
        bridge=bridge,
        switch_state=switch_state,
        color_channel=switch_toggle_color,
        resends=resends,
        resend_gap=resend_gap,
        switch_settle=switch_settle,
    )
    auto_toggles += 1
    continue

# Finalize only red routing after red trigger confirmation.
if switch_state.red_toggled != target_red:
    await send_switch_toggle(
        bridge=bridge,
        switch_state=switch_state,
        color_channel=gc.COLOR_RED,
        resends=resends,
        resend_gap=resend_gap,
        switch_settle=switch_settle,
    )

await asyncio.wait_for(
```

Listing 4.8: Auszug: Discard-Phase nach vollständiger Keep-Bestätigung

```
remaining = total_count - shots_sent
if remaining > 0:
    if switch_state.red_toggled != discard_exit_red_toggled:
        await send_switch_toggle(
            bridge=bridge,
            switch_state=switch_state,
            color_channel=switch_toggle_color,
            resends=resends,
            resend_gap=resend_gap,
            switch_settle=switch_settle,
        )

    await bridge.send_periodic(
        status=gc.STATUS_CANNON,
        color_channel=gc.COLOR_RED,
        count=remaining,
        gap=cannon_gap,
        resends=resends,
        resend_gap=resend_gap,
    )
```


5 Mit Gravitrax programmieren lernen

- Als basics: Nein - Asynchrone Programmierung: Auch eher nicht weil nur callback ding und das ist durch async/await eigentlich nicht mehr so ein Ding

6 Fazit und Ausblick

6.1 Zusammenfassung

Suspendisse vitae elit. Aliquam arcu neque, ornare in, ullamcorper quis, commodo eu, libero. Fusce sagittis erat at erat tristique mollis. Maecenas sapien libero, molestie et, lobortis in, sodales eget, dui. Morbi ultrices rutrum lorem. Nam elementum ullamcorper leo. Morbi dui. Aliquam sagittis. Nunc placerat. Pellentesque tristique sodales est. Maecenas imperdiet lacinia velit. Cras non urna. Morbi eros pede, suscipit ac, varius vel, egestas non, eros. Praesent malesuada, diam id pretium elementum, eros sem dictum tortor, vel consectetur odio sem sed wisi.

6.2 Ausblick

Sed feugiat. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Ut pellentesque augue sed urna. Vestibulum diam eros, fringilla et, consectetur eu, nonummy id, sapien. Nullam at lectus. In sagittis ultrices mauris. Curabitur malesuada erat sit amet massa. Fusce blandit. Aliquam erat volutpat. Aliquam euismod. Aenean vel lectus. Nunc imperdiet justo nec dolor.

A Anhang

Hier koennen zusaetzliche Materialien ergaenzt werden, z. B. grosse Tabellen, Frageboegen, Screenshots oder weitere Auswertungen.